

PocketSmart AI: Your Smart Budget & Recommendation Assistant

Project Description

This project is developed to provide an effective and user-friendly solution to a real-world problem. The main purpose of the project is to reduce manual work, improve accuracy, save time, and make the overall process easier and more efficient. The proposed system is designed with useful features that help users perform their required tasks in a simple and organized manner. It includes different modules to manage, process, and store information effectively. The project can be developed using suitable technologies such as programming languages, frameworks, databases, and other software tools. The system focuses on providing better performance, data security, easy accessibility, and a user-friendly interface. It can be useful for individuals, organizations, educational institutions, businesses, or other relevant users depending on the project requirements. The proposed system also has scope for future enhancement by adding new features and improving its functionality. Overall, this project aims to provide a reliable, efficient, and practical solution to the identified problem.

Scenario 1: Home Interior Planning with Smart Budget Allocation

Home Interior Planning with Smart Budget Allocation is a system designed to help users plan and manage their home interior projects efficiently within a specified budget. The system allows users to enter their total budget, select different rooms such as the living room, bedroom, kitchen, and dining area, and choose their preferred interior style and requirements. Based on the available budget, the system intelligently allocates the amount for different categories such as furniture, lighting, flooring, wall decoration, curtains, and other interior items. It helps users understand how much they can spend on each category while avoiding unnecessary expenses. The system can also provide suitable recommendations based on the user's preferences, room size, and budget. By organizing the expenses and providing a clear budget breakdown, the project helps users make better planning decisions and complete their interior design within the available financial limit. The main objective of this scenario is to combine interior planning with smart budget management to provide a convenient, cost-effective, and user-friendly solution for home decoration.

Scenario 2: AI-Based Party Budget Planning

AI-Based Party Budget Planning is a system that helps users plan a party efficiently within a fixed budget. Users can enter details such as the number of guests, party type, location, food preferences, and total budget. The AI analyzes these requirements and suggests suitable allocations for food, decoration, venue, entertainment, and other expenses. This helps users control their spending, avoid unnecessary costs, and organize the party according to their budget. The system provides a simple and smart way to create a well-planned and cost-effective party.

Scenario 3: Jewelry Recommendations for Occasions

Jewelry Recommendations for Occasions is a system that helps users choose suitable jewelry based on the type of occasion, outfit, personal style, and budget. Users can select occasions such as weddings, parties, festivals, or casual events, and the system suggests matching jewelry options. It helps users save time and make better choices by providing personalized and

occasion-based recommendations.

Architecture Overview

The architecture of the system describes how different components work together to provide the required services. It generally includes the user interface, application or backend layer, database, and AI or recommendation module. The user interacts with the system through the interface, while the backend processes the input and communicates with the database and AI module. The processed results are then returned to the user in a simple and understandable format.

Core Technologies

- **Flask:** Backend service layer for request handling, routing, and session control.
- **Gemini 1.5 Flash Pro:** AI model used for understanding budget context, analyzing images, generating structured suggestions, and recommending platform-specific products.
- **Third-party APIs (Amazon, Flipkart, IKEA, Zomato, etc.):** Product and service data sourcing.
- **Frontend (HTML/CSS + JS):** Interactive user interface for input collection and AI output display.

Component-Wise Architecture

Component Description

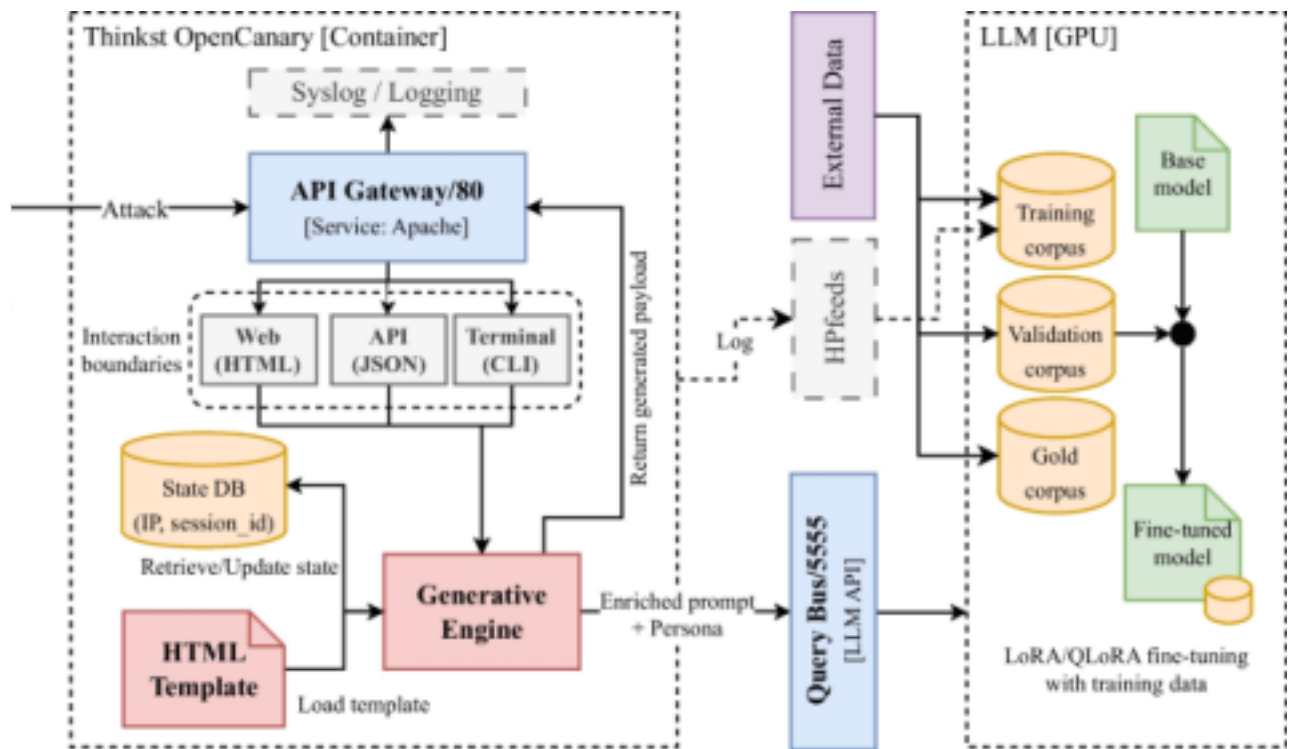
Frontend UI Web interface built with HTML/CSS allows users to input budgets, preferences, and images. Displays tailored AI-driven recommendations for each category.

Flask Backend

(/generate-home, /generate-party, /generate-jewelry), and handles communication between UI and Gemini AI.

Processes inputs using Gemini 1.5 Flash Pro to:

Gemini AI Layer
Manages user sessions, routes



Pre-requisites

1. IBM Cloud Account Setup: [IBM Cloud Console](#)
2. Watsonx Access and Configuration: [Watsonx Overview](#)
3. Python Basics: [Python.org](#)
4. FastAPI Framework: [FastAPI Docs](#)

Project Workflow

1. Gemini AI Setup & Initialization

- **Activity 1.1:** Set up Google Cloud access (or platform) with Gemini 1.5 Flash Pro API access.
- **Activity 1.2:** Configure API keys, usage limits, and test integration for multimodal support (text + image).
- **Activity 1.3:** Fine-tune prompts for budget interpretation and product recommendation generation.

2. Core Functionality Development

- **Activity 2.1:** Design Flask-based backend structure for routing, session control, and planner modules.

- **Activity 2.2:** Create endpoints for Home Planner, Party Planner, and Jewelry Planner.

- **Activity 2.3:** Integrate Gemini prompts and templates for each planner with support for text + optional image inputs.

3. Backend Implementation (app.py)

- **Activity 3.1:** Define Flask routes like `/generate-home`, `/generate-party`, `/generate-jewelry`.
- **Activity 3.2:** Organize backend using modular structure:
 - `routes/`: Endpoints
 - `services/`: Recommendation logic & AI calls
 - `models/`: Input/output schemas
- **Activity 3.3:** Setup CORS headers for frontend communication and implement basic session management.
- **Activity 3.4:** Include mock API calls or simulated scraping from Amazon, IKEA, Zomato, etc.

4. UI Development

- **Activity 4.1:** Build responsive HTML forms for:
 - Home Interior Planner (room details, quantities)
 - Party Planner (event type, budget, guests)
 - Jewelry Planner (budget, occasion, outfit image upload)
- **Activity 4.2:** Add dynamic result display and formatting for recommendations using JavaScript.

5. Testing & Optimization

- **Activity 5.1:** Perform testing with real-world use cases across all three planners.
- **Activity 5.2:** Evaluate Gemini response quality, budget adherence, and platform accuracy.
- **Activity 5.3:** Optimize prompts and add validations to prevent input edge-case failures.
- **Activity 5.4:** Add fallback/default recommendations when AI returns insufficient results.

MILESTONE 1: Gemini AI Initialization

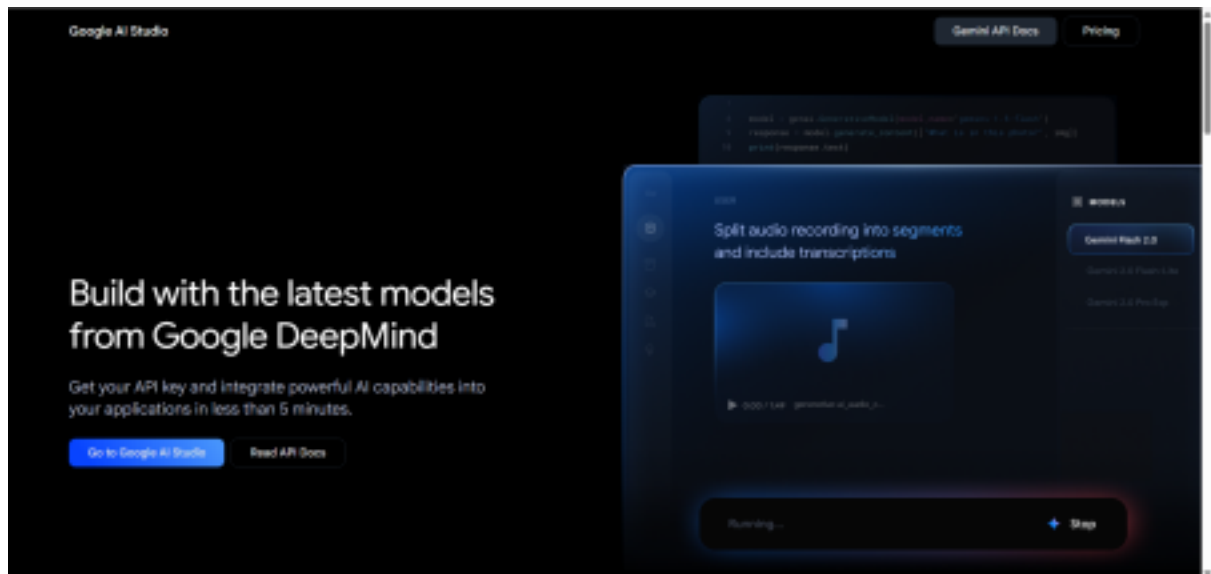
In this foundational stage, the objective is to establish and configure the core AI infrastructure required to power **PocketSmart AI**. The platform relies on **Gemini 1.5 Flash Pro**—Google's multimodal foundation model—to interpret user inputs, process budgets, and generate personalized recommendations. This milestone ensures that the development environment is equipped with authenticated access to Gemini's APIs and is capable of

performing both text-based reasoning and image analysis (specifically for the Jewelry Planner). Proper completion of this setup is essential, as it enables all AI-driven functionalities like budget understanding, category-wise recommendations, and multimodal analysis to operate seamlessly.

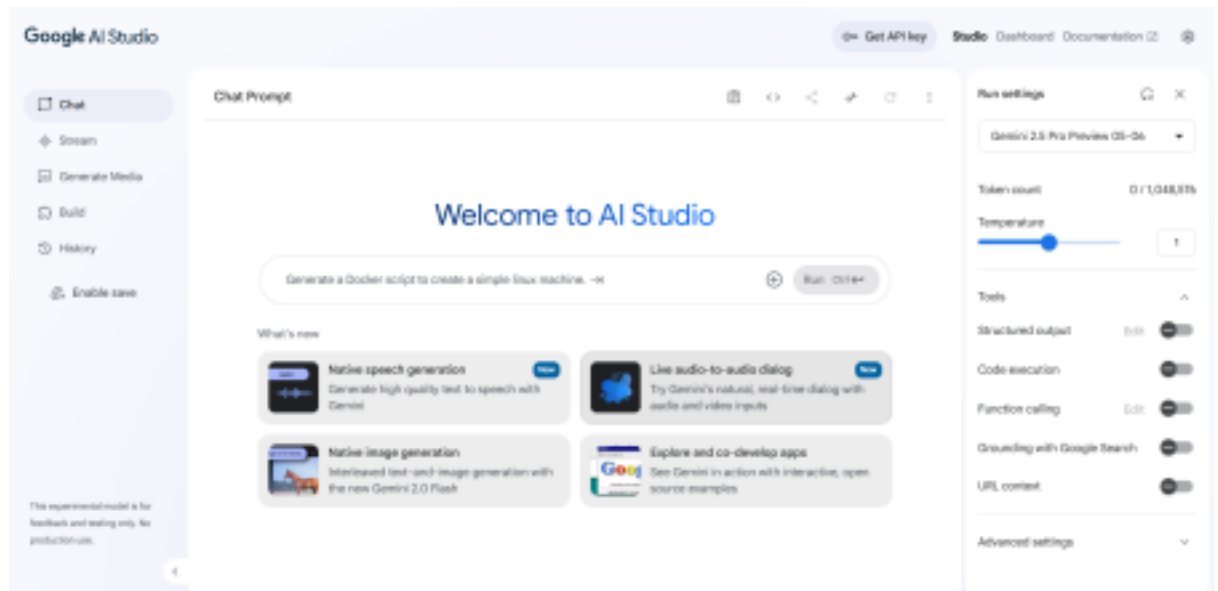
- **Activity 1.1: Set up Google Cloud account and enable Gemini**

API ● Sign in or create a Google Cloud account via

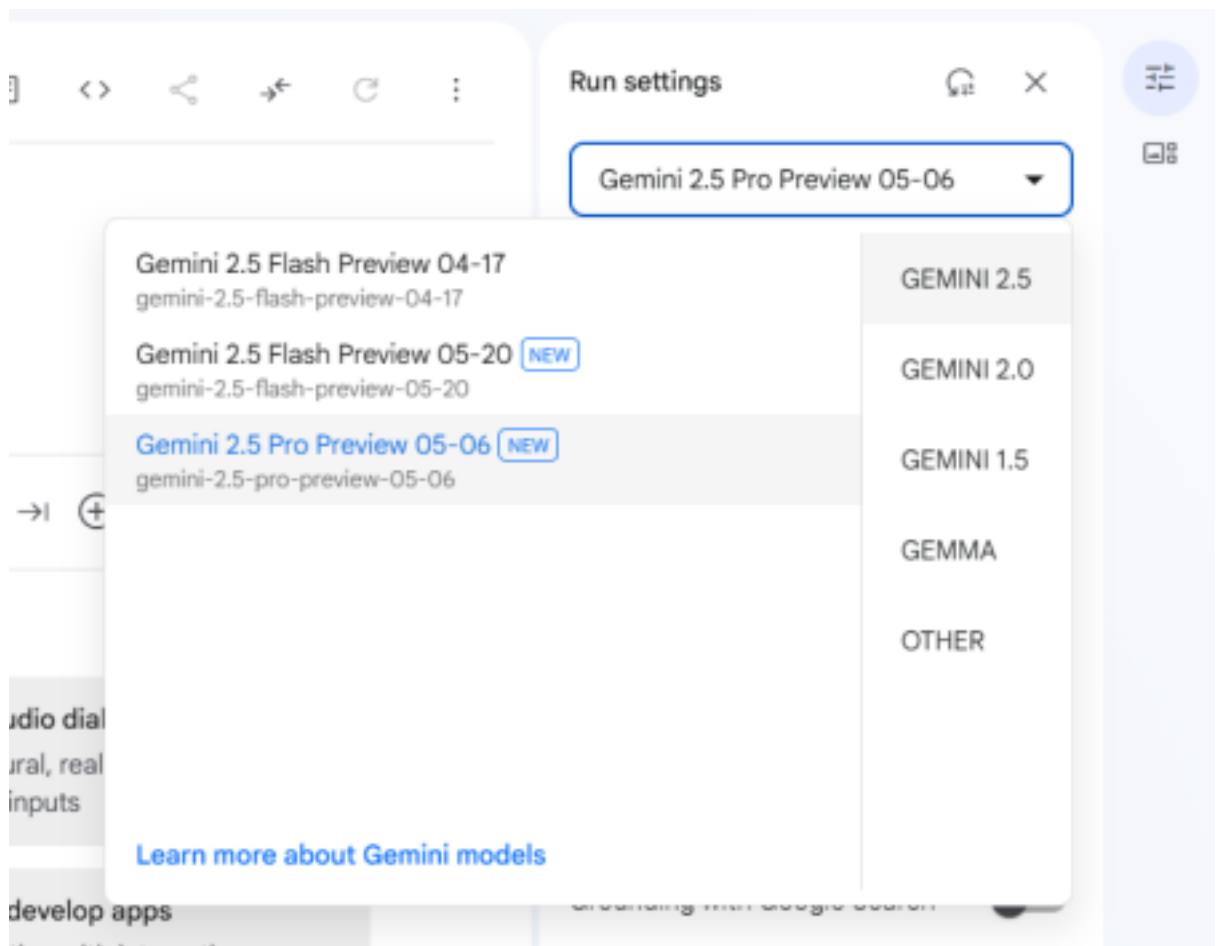
console.cloud.google.com.



- Accept all terms and conditions and click on Get API key



- You will get trending models here you can select the model here or you can set the model by go to the API Documentation.



Activity 1.2: Configure access and permissions

- You will get interface like this and now click on Create API key



- Once click on API key you will the api key and make sure to copy the API key
- **Activity 1.3: Validate Gemini API connectivity**
 - Make a sample prompt call using Python to confirm integration.

```

from google import genai

client =
genai.Client(api_key="YOUR_API_KEY")

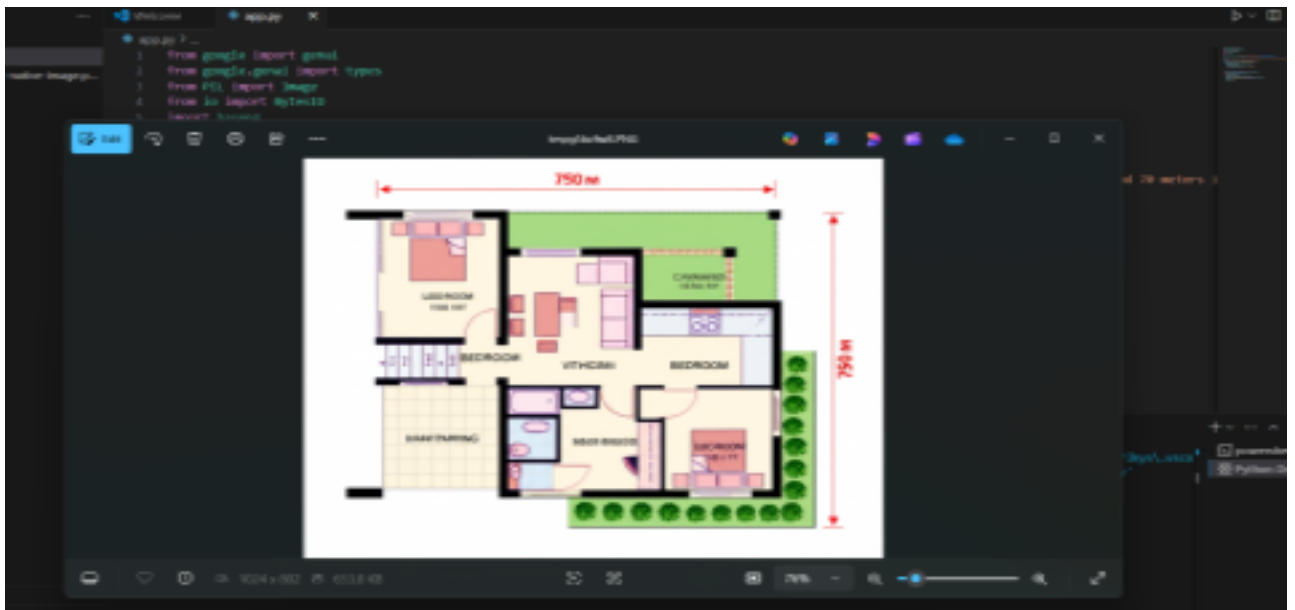
prompt = "Hello Gemini! Please
confirm that the AI integration is
working."

response =
client.models.generate_content(
    model="gemini-2.5-flash",
    contents=prompt
)

print("AI Response:")
print(response.text)

```

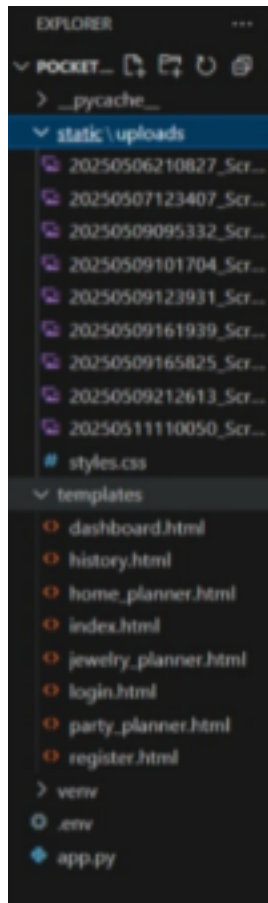
- Test both text-only and image + text prompts to ensure multimodal Compatibility.



MILESTONE 2: Core Functionalities Development

This milestone focuses on building the **core backend intelligence** of **PocketSmart AI** using the Flask framework and Google's **Gemini 1.5 Flash Pro** foundation model. The primary goal is to develop a robust and modular system capable of accepting user budgets (and optional image inputs), intelligently interpreting the context, and returning domain-specific, platform-aware recommendations. The backend handles prompt generation, API communication with Gemini, modular planning logic (for Home, Party, and Jewelry), and optional fallback strategies when product matches are limited. All core logic is abstracted into a utility service file ([gemini_utils.py](#)) that handles prompt orchestration, budget formatting, domain segmentation, and image analysis for multimodal tasks.

File Explorer Structure



Activity 2.1: Fast API Instilization and import libraries

- Import all required libraries for the project

```
# Import required libraries

import math
import random
import datetime

print("Required libraries imported
successfully!")

# Using math library
print("Square root of 25:",
math.sqrt(25))

# Using random library
print("Random number:",
random.randint(1, 10))

# Using datetime library
today = datetime.date.today()
print("Today's date:", today)
```

- Fast API Initialization

```
# FastAPI app initialization
app = FastAPI(title="PocketSmart: AI Budget Planner")

SECRET_KEY = os.getenv("SECRET_KEY", "your_secret_key")
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30
```

- Load .env environment

```
goal = float(input("Enter your
savings goal: ₹"))
monthly_saving = float(input("Enter
monthly saving amount: ₹"))

months = goal / monthly_saving

print("\n==== SAVINGS PLAN =====")
print("Savings Goal      : ₹", goal)
print("Monthly Saving    : ₹",
monthly_saving)
print("Months Required   :",
round(months, 1))
```

Activity 2.2: Main Functionality code routings

- `/generate-home`: Generates home interior recommendations (furniture, decor, lighting)

```
@app.get("/home-planner", response_class=HTMLResponse)
async def home_planner(request: Request, current_user: User = Depends(get_current_active_user)):
    """home budget planner page"""
    return templates.TemplateResponse("home_planner.html", {"request": request, "user": current_user})
```

- get home_recommendations by using the function

```

food = float(input("Enter food
expense: ₹"))
travel = float(input("Enter travel
expense: ₹"))
shopping = float(input("Enter
shopping expense: ₹"))

total = food + travel + shopping

print("\n==== EXPENSE ANALYSIS
====")
print("Food Expense      : ₹", food)
print("Travel Expense    : ₹",
travel)
print("Shopping Expense : ₹",
shopping)
print("Total Expense     : ₹", total)

if food > travel and food >
shopping:
    print("Highest Expense: Food")
elif travel > shopping:
    print("Highest Expense: Travel")
else:
    print("Highest Expense:
Shopping")

```

- linkingthewebsiteslikeamazon,flipkar,ikeaect.... Togetheproducts

```

days = int(input("Enter number of
days: "))
total = 0

for i in range(1, days + 1):
    expense = float(input("Enter
expense for day " + str(i) + ": ₹"))
    total = total + expense

average = total / days

print("\n==== DAILY EXPENSE REPORT
====")
print("Number of Days : ", days)
print("Total Expense   : ₹", total)
print("Average Expense: ₹",
round(average, 2))

```

- `/generate-party`: Suggestscatering, venue, anddecorationoptionsforevents

```
budget = float(input("Enter your
monthly budget: ₹"))
expense = float(input("Enter your
total expense: ₹"))

balance = budget - expense

print("\n==== BUDGET STATUS =====")
print("Monthly Budget : ₹", budget)
print("Total Expense : ₹", expense)
print("Balance : ₹", balance)

if balance > 0:
    print("Status: Budget is under
control.")
elif balance == 0:
    print("Status: Budget is fully
used.")
else:
    print("Status: Budget
exceeded.")
```

- get home_recommendations by using the function

```
print("==== POCKETSMART AI =====")

room = input("Enter room type: ")
style = input("Enter your preferred
style: ")
budget = int(input("Enter your
budget: ₹"))

if room.lower() == "living room":
    if style.lower() == "modern":
        print("Reconnended: Modern
Sofa, TV Unit and Floor Lamp")
    else:
        print("Reconnended: Classic
Sofa, Curtains and Wall Decor")

elif room.lower() == "bedroom":
    if style.lower() == "modern":
        print("Reconnended: Platform
Bed, Wardrobe and LED Lamp")
    else:
```

```

else:
    print("Recommended: Wooden
    Bed, Curtains and Bedside Table")

elif room.lower() == "study room":
    if style.lower() == "modern":
        print("Recommended: Study
        Table, Ergonomic Chair and Desk
        Lamp")
    else:
        print("Recommended: Wooden
        Table, Bookshelf and Table Lamp")

else:
    print("No recommendation
    available.")

print("Budget: ₹", budget)

```

- linking the websites like amazon, flipkar, ikea etc.... To get the products

```

print("==== POCKETSMART AI =====")
print("SMART HOME RECOMMENDATION")

room = input("Enter room type: ")
style = input("Enter your preferred style: ")
budget = int(input("Enter your budget: ₹"))

print("\n==== RECOMMENDATION =====")

if room.lower() == "living room":
    if style.lower() == "modern":
        print("Recommended: Modern Sofa, TV Unit and Table Lamp")
    else:
        print("Recommended: Classic Sofa, Curtains and Wall Decor")

```

```

elif room.lower() == "bedroom":
    if style.lower() == "modern":
        print("Recommended: Platform Bed, Wardrobe and LED Lamp")
    else:
        print("Recommended: Wooden Bed, Curtains and Bedside Table")

elif room.lower() == "study room":
    if style.lower() == "modern":
        print("Recommended: Study Table, Ergonomic Chair and Desk Lamp")
    else:
        print("Recommended: Wooden Table, Bookshelf and Table Lamp")

else:
    print("No recommendation available.")

print("\nBudget considered: ₹", budget)
print("Home recommendation generated successfully!")

```

- using the if condition to categorize the situation based products

```

print("==== POCKETSMART AI ====")

room = input("Enter room type: ")
style = input("Enter your preferred style: ")
budget = int(input("Enter your budget: ₹"))

if room.lower() == "living room":
    if style.lower() == "modern":
        print("Recommended: Modern Sofa, TV Unit and Floor Lamp")
    else:
        print("Recommended: Classic Sofa, Curtains and Wall Decor")

elif room.lower() == "bedroom":
    if style.lower() == "modern":
        print("Recommended: Platform Bed, Wardrobe and LED Lamp")
    else:
        print("Recommended: Wooden Bed, Curtains and Bedside Table")

elif room.lower() == "study room":
    if style.lower() == "modern":
        print("Recommended: Study Table, Ergonomic Chair and Desk Lamp")
    else:
        print("Recommended: Wooden Table, Bookshelf and Table Lamp")

else:
    print("No recommendation available.")

print("Budget: ₹", budget)

```

- `/generate-jewelry`: Accepts text + image inputs to return style-matched jewelry options.

```

• async def jewelry_planner(request: Request, current_user:
  UserInDB = Depends(get_current_active_user)):
  """Jewelry budget planner page"""
  return templates.TemplateResponse("jewelry_planner.html",
    {"request": request, "user": current_user})

```

- `async def jewelry_planner(request: Request, current_user: UserInDB = Depends(get_current_active_user)):`
- `"""Jewelry budget planner page"""`
- `return templates.TemplateResponse("jewelry_planner.html", {"request": request, "user": current_user})`
- get home_recommendations by using the function

```
def get_jewelry_recommendations(budget_input: JewelryBudgetInput, image_path: Optional[str] = None) -> dict:
    """Generate jewelry recommendations based on uploaded dress and budget in INR (India-specific)"""
    try:
        # Base prompt with INR context
        base_prompt = f"""
        I need jewelry recommendations for India with a total budget of ₹{budget_input.total_budget:.2f}.

        Occasion: {budget_input.occasion}
        Preferences: {budget_input.preferences or "Not specified"}
        Provide only India-relevant styles, availability, and price ranges in INR.
        """

        if image_path:
            # Include image-based outfit analysis
            img = Image.open(image_path)
            prompt = base_prompt + """

            An image of the outfit is uploaded. Suggest jewelry that complements it, considering color, design, and occasion appropriateness.

            Format the output as JSON:
            {
                "outfit_analysis": {
                    "colors": [],
                    "style": "",
                    "formality": ""
                },
                "total_budget": 0.0,
                "jewelry_recommendations": [
                    {
                        "item_type": "",
                        "description": "",
                        "style": "",
                        "estimated_price": 0.0,
                        "search_terms": ""
                    }
                ]
            },
            """
    except Exception as e:
        print(f"Error in get_jewelry_recommendations: {e}")
        return {}
```

```
def get_party_recommendations(budget_input: PartyBudgetInput) -> dict:
    """Generate party recommendations based on budget in INR (India-specific)"""
    try:
        # Base prompt with INR context
        base_prompt = f"""
        I need party recommendations for India with a total budget of ₹{budget_input.total_budget:.2f}.

        Venue: {budget_input.venue or "Not specified"}
        Occasion: {budget_input.occasion}
        Preferences: {budget_input.preferences or "Not specified"}
        Provide only India-relevant venues, availability, and price ranges in INR.
        """

        if image_path:
            # Include image-based outfit analysis
            img = Image.open(image_path)
            prompt = base_prompt + """

            An image of the outfit is uploaded. Suggest jewelry that complements it, considering color, design, and occasion appropriateness.

            Format the output as JSON:
            {
                "venue_suggestions": [
                    {
                        "name": "",
                        "type": "",
                        "capacity": 0,
                        "estimated_cost": 0.0,
                        "search_terms": ""
                    }
                ],
                "remaining_budget": 0.0,
                "additional_suggestions": []
            }
            """
    except Exception as e:
        print(f"Error in get_party_recommendations: {e}")
        return {}

    response = model.generate_content(prompt)
    result = extract_json_from_response(response.text)

    # Create INR calculation table
    result["calculation_table_inr"] = []
    categories = {}

    for category in result.get("budget_breakdown", []):
        cat_name = category.get("category", "Misc")
        cat_allocation = category.get("allocation", 0)

        if cat_name not in categories:
            categories[cat_name] = {
                "category": cat_name,
                "items_count": 0,
            }
```

- linkingthewebsiteslikeamazon,flipkar,ikeaect....
Togetheproducts

```
def get_party_recommendations(budget_input: PartyBudgetInput) -> dict:
    for item in category.get("items", []):
        categories[cat_name]["items_count"] += 1
        categories[cat_name]["total_cost"] += item.get("estimated_price", 0)

    if result["total_budget"] > 0:
        categories[cat_name]["percentage_of_budget"] = (categories[cat_name]["total_cost"] / result["total_budget"]) * 100

    for cat_data in categories.values():
        result["calculation_table_inr"].append(cat_data)

    # Define relevant shopping platforms for each category
    category_platforms = {
        "venue": ["google", "booking", "makemytrip", "oyorooms", "nobroker"],
        "catering": ["swiggy", "zomato"],
        "food": ["swiggy", "zomato", "bigbasket", "amazon", "flipkart"],
        "drinks": ["swiggy", "zomato", "bigbasket", "amazon", "flipkart"],
        "decoration": ["amazon", "flipkart", "meesho", "myntra"],
        "entertainment": ["bookmyshow", "amazon", "flipkart"],
        "gifts": ["amazon", "flipkart", "myntra", "meesho"],
        "photography": ["google", "amazon", "flipkart"],
        "music": ["amazon", "flipkart", "bookmyshow"],
        "games": ["amazon", "flipkart"],
        "accessories": ["amazon", "flipkart", "myntra", "meesho"],
        "transportation": ["makemytrip", "google"],
        "return_gifts": ["amazon", "flipkart", "myntra", "meesho"]
    }

    # Default platforms if category is not in our predefined list
    default_platforms = ["amazon", "flipkart", "google"]

    # Add shopping links for each item
    for category in result.get("budget_breakdown", []):
        cat_name = category.get("category", "").lower()
        relevant_platforms = category_platforms.get(cat_name, default_platforms)
```

- Using the if condition to categorize the situation based project

```
def get_party_recommendations(budget_input: PartyBudgetInput) -> dict:
    for category in result.get("budget_breakdown", []):
        cat_name = category.get("category", "").lower()
        relevant_platforms = category_platforms.get(cat_name, default_platforms)

        for item in category.get("items", []):
            search_terms = item.get("search_terms", "")
            if search_terms:
                item["shopping_links"] = {}

                # Only add relevant platforms for this category
                if "amazon" in relevant_platforms:
                    item["shopping_links"]["amazon"] = f"https://www.amazon.in/s?k={urllib.parse.quote_plus(search_terms)}"
                if "flipkart" in relevant_platforms:
                    item["shopping_links"]["flipkart"] = f"https://www.flipkart.com/search?q={urllib.parse.quote_plus(search_terms)}"
                if "bigbasket" in relevant_platforms:
                    item["shopping_links"]["bigbasket"] = f"https://www.bigbasket.com/ps/?q={urllib.parse.quote_plus(search_terms)}"
                if "swiggy" in relevant_platforms:
                    item["shopping_links"]["swiggy"] = f"https://www.swiggy.com/search?query={urllib.parse.quote_plus(search_terms)}"
                if "zomato" in relevant_platforms:
                    item["shopping_links"]["zomato"] = f"https://www.zomato.com/search?q={urllib.parse.quote_plus(search_terms)}"
                if "bookmyshow" in relevant_platforms:
                    item["shopping_links"]["bookmyshow"] = f"https://in.bookmyshow.com/search?q={urllib.parse.quote_plus(search_terms)}"
                if "myntra" in relevant_platforms:
                    item["shopping_links"]["myntra"] = f"https://www.myntra.com/search?q={urllib.parse.quote_plus(search_terms)}"
                if "meesho" in relevant_platforms:
                    item["shopping_links"]["meesho"] = f"https://www.meesho.com/search?q={urllib.parse.quote_plus(search_terms)}"
                if "google" in relevant_platforms:
                    item["shopping_links"]["google"] = f"https://www.google.com/search?q={urllib.parse.quote_plus(search_terms)}"
                if "booking" in relevant_platforms:
                    item["shopping_links"]["booking"] = f"https://www.booking.com/search.html?ss={urllib.parse.quote_plus(search_terms)}"
                if "makemytrip" in relevant_platforms:
                    item["shopping_links"]["makemytrip"] = f"https://www.makemytrip.com/hotels/hotel-listing/?searchText={urllib.parse.quote_plus(search_terms)}"
                if "oyorooms" in relevant_platforms:
                    item["shopping_links"]["oyorooms"] = f"https://www.oyorooms.com/search/?location={urllib.parse.quote_plus(search_terms)}"
                if "nobroker" in relevant_platforms:
                    item["shopping_links"]["nobroker"] = f"https://www.nobroker.in/property/search?searchTerm={urllib.parse.quote_plus(search_terms)}"
```

```

# Add search links for venue suggestions
venue_platforms = ["google", "booking", "makemytrip", "oyorooms", "nobroker"]
for venue in result.get("venue_suggestions", []):
    search_terms = venue.get("search_terms", "")
    if search_terms:
        venue["search_links"] = {}
        if "google" in venue_platforms:
            venue["search_links"]["google"] = f"https://www.google.com/search?q={urllib.parse.quote_plus(search_terms)}"
        if "booking" in venue_platforms:
            venue["search_links"]["booking"] = f"https://www.booking.com/search.html?ss={urllib.parse.quote_plus(search_terms)}"
        if "makemytrip" in venue_platforms:
            venue["search_links"]["makemytrip"] = f"https://www.makemytrip.com/hotels/hotel-listing/?searchText={urllib.parse.quote_plus(search_terms)}"
        if "oyorooms" in venue_platforms:
            venue["search_links"]["oyorooms"] = f"https://www.oyorooms.com/search/?location={urllib.parse.quote_plus(search_terms)}"
        if "nobroker" in venue_platforms:
            venue["search_links"]["nobroker"] = f"https://www.nobroker.in/property/search?searchTerm={urllib.parse.quote_plus(search_terms)}"

return result

except Exception as e:
    raise HTTPException(500, f"Error generating recommendations: {str(e)}")

```

- /generate-jewelry: Accepts text + image inputs to return style-matched jewelry option.

```

async def jewelry_planner(request: Request, current_user:
    UserInDB = Depends(get_current_active_user)):
    """Jewelry budget planner page"""
    return templates.TemplateResponse("jewelry_planner.html",
        {"request": request, "user": current_user})

```

- Get home_recommendations by using the function

```

def get_jewelry_recommendations(budget_input: JewelryBudgetInput, image_path: Optional[str] = None) -> dict:
    """Generate jewelry recommendations based on uploaded dress and budget in INR (India-specific)"""
    try:
        # Base prompt with INR context
        base_prompt = f"""
        I need jewelry recommendations for India with a total budget of ₹(budget_input.total_budget:.2f).

        Occasion: {budget_input.occasion}
        Preferences: {budget_input.preferences or "Not specified"}
        Provide only India-relevant styles, availability, and price ranges in INR.
        """

        if image_path:
            # Include image-based outfit analysis
            img = Image.open(image_path)
            prompt = base_prompt + """

            An image of the outfit is uploaded. Suggest jewelry that complements it, considering color, design, and occasion appropriateness.

            Format the output as JSON:
            {
                "outfit_analysis": {
                    "colors": [],
                    "style": "",
                    "formality": ""
                },
                "total_budget": 0.0,
                "jewelry_recommendations": [
                    {
                        "item_type": "",
                        "description": "",
                        "style": "",
                        "estimated_price": 0.0,
                        "search_terms": ""
                    }
                ]
            }
            """

```

```
def get_jewelry_recommendations(budget_input: JewelryBudgetInput, image_path: Optional[str] = None) -> dict:
    """
    styling_tips: []
    """
    Make sure prices are in INR and stay within budget.
    Include Indian-friendly search terms for shopping.
    """
    response = model.generate_content([prompt, img])
else:
    # Only text input
    prompt = base_prompt + """

    Format the output as JSON:
    {
        "total_budget": 0.0,
        "jewelry_recommendations": [
            {
                "item_type": "",
                "description": "",
                "style": "",
                "estimated_price": 0.0,
                "search_terms": ""
            }
        ],
        "remaining_budget": 0.0,
        "styling_tips": []
    }

    Keep prices in INR and relevant to Indian brands.
    """
    response = model.generate_content(prompt)

    result = extract_json_from_response(response.text)

    # Add shopping links for each item (India-specific)
    for item in result.get("jewelry_recommendations", []):
```

- Linking the websites like amazon,flipkar,ikea etc....
To get the products

```
def get_jewelry_recommendations(budget_input: JewelryBudgetInput, image_path: Optional[str] = None) -> dict:
    # Add shopping links for each item (India-specific)
    for item in result.get("jewelry_recommendations", []):
        search_terms = item.get("search_terms", "")
        if search_terms:
            item["shopping_links"] = {
                "amazon": f"https://www.amazon.in/s?k={urllib.parse.quote_plus(search_terms)}",
                "flipkart": f"https://www.flipkart.com/search?q={urllib.parse.quote_plus(search_terms)}",
                "bluestone": f"https://www.bluestone.com/search.html?query={urllib.parse.quote_plus(search_terms)}",
                "tanishq": f"https://www.tanishq.co.in/search?q={urllib.parse.quote_plus(search_terms)}",
                "caratlane": f"https://www.caratlane.com/search?q={urllib.parse.quote_plus(search_terms)}",
                "melorra": f"https://www.melorra.com/search?q={urllib.parse.quote_plus(search_terms)}",
                "meesho": f"https://www.meesho.com/search?q={urllib.parse.quote_plus(search_terms)}"
            }

    return result

except Exception as e:
    raise HTTPException(500, f"Error generating recommendations: {str(e)}")
```

Activity 2.3: Login and Register page routings

- Login page

/login: Authenticates user credentials and initiates a session upon successful login.

```

@app.get("/login", response_class=HTMLResponse)
async def login_page(request: Request):
    """Serve the login page"""
    # Check if user is already logged in
    try:
        token = await get_token(request)
        if token:
            user = await get_current_user(request, token)
            if user:
                return RedirectResponse(url="/dashboard", status_code=status.HTTP_302_FOUND)
    except Exception:
        pass

    return templates.TemplateResponse("login.html", {"request": request})

```

- Register Page

/register: Handles new user registration by accepting and securely storing user credentials.

```

@app.get("/register", response_class=HTMLResponse)
async def register_page(request: Request):
    """Serve the registration page"""
    # Check if user is already logged in
    try:
        token = await get_token(request)
        if token:
            user = await get_current_user(request, token)
            if user:
                return RedirectResponse(url="/dashboard", status_code=status.HTTP_302_FOUND)
    except Exception:
        pass

    return templates.TemplateResponse("register.html", {"request": request})

```

- Logout Page

/logout: Terminates the current user session and redirects to the

```

@app.post("/logout")
async def logout(request: Request):
    """Logout user by blacklisting their token and clearing session"""
    token = await get_token(request)

    if token:
        # Add token to blacklist
        blacklisted_tokens.add(token)

        try:
            # Extract username from token to remove from active sessions
            payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
            username = payload.get("sub")
            if username and username in active_sessions:
                del active_sessions[username]
        except JWTError:
            pass

    # Use RedirectResponse instead of JSONResponse for proper redirection
    response = RedirectResponse(url="/login", status_code=status.HTTP_302_FOUND)
    response.delete_cookie(key="access_token")

```

Activity 2.4: Ensure modular code structure

- **/token**: Issues a secure JWT token after successful authentication, enabling authorized access to protected API routes.

```

@app.post("/token", response_model=Token)
async def login_for_access_token(form_data: OAuth2PasswordRequestForm = Depends()):
    """Login endpoint to get access token"""
    user = authenticate_user(users_db, form_data.username, form_data.password)
    if not user:
        raise HTTPException(
            status_code=401,
            detail="Incorrect username or password",
            headers={"WWW-Authenticate": "Bearer"},
        )

    # Create access token
    access_token_expires = timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    access_token = create_access_token(
        data={"sub": user.username}, expires_delta=access_token_expires
    )

    # Create or update session for the user
    existing_user_data = {}
    if user.username in active_sessions:
        existing_user_data = active_sessions[user.username].user_data
        # If there's an old token, blacklist it
        old_token = active_sessions[user.username].token
        blacklisted_tokens.add(old_token)

    active_sessions[user.username] = UserSession(
        username=user.username,
        login_time=datetime.utcnow(),
        last_activity=datetime.utcnow(),
        token=access_token,
        user_data=existing_user_data
    )

    # Return response with cookie
    response = JSONResponse(content={"access_token": access_token, "token_type": "bearer"})

    response.set_cookie(
        key="access_token",
        value=access_token, # Store token directly without Bearer prefix
        httponly=True,
        max_age=ACCESS_TOKEN_EXPIRE_MINUTES * 60,
        samesite="lax"
    )

    return response

```

- **/session-info**: Retrieves metadata about the current user session, such as user ID and login status.
- **/session-data**: Returns detailed session-specific data used for personalization and recommendation tracking.

```

print("==== POCKETSMART AI ====")
print("SMART HOME RECOMMENDATION")

room = input("Enter room type: ")
style = input("Enter your preferred
style: ")
budget = int(input("Enter your
budget: ₹"))

print("\n==== RECOMMENDATION
====")

if room.lower() == "living room":
    if style.lower() == "modern":
        print("Recommended: Modern
Sofa, TV Unit and Floor Lamp")
    else:
        print("Recommended: Classic
Sofa, Curtains and Wall Decor")

elif room.lower() == "bedroom":
    if style.lower() == "modern":
        print("Recommended: Platform
Bed, Wardrobe and LED Lamp")
    else:

```

```

        print("Recommended: Classic
Sofa, Curtains and Wall Decor")

elif room.lower() == "bedroom":
    if style.lower() == "modern":
        print("Recommended: Platform
Bed, Wardrobe and LED Lamp")
    else:
        print("Recommended: Wooden
Bed, Curtains and Bedside Table")

elif room.lower() == "study room":
    if style.lower() == "modern":
        print("Recommended: Study
Table, Ergonomic Chair and Desk
Lamp")
    else:
        print("Recommended: Wooden
Table, Bookshelf and Table Lamp")

else:
    print("No recommendation
available.")

print("\nBudget considered: ₹",
budget)
print("Home recommendation generated
successfully!")

```

MILESTONE 3: Backend – FastAPI Integration (**main.py**)

Milestone 3 focuses on integrating the backend using FastAPI in the main.py file. It provides API endpoints to handle user requests, process data, and connect the frontend with the AI service. This integration helps the PocketSmart AI application communication efficiently and Provide smart budget recommendations.

- **Activity 3.1: Define FastAPI Routes**

- Home Planner (**/generate-home**): Accepts home-related preferences and Budget

```
@app.post("/home-budget")
✓ async def plan_home_budget(
    budget_input: HomeBudgetInput,
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Generate home budget recommendations"""
    # Store last budget planning in session data
    ✓ if current_user.username in active_sessions:
        ✓ active_sessions[current_user.username].user_data["last_home_budget"] = {
            "timestamp": datetime.datetime.now().isoformat(),
            "budget": budget_input.total_budget,
            "requirements": {
                "lights": budget_input.num_lights,
                "fans": budget_input.num_fans,
                "furniture": budget_input.num_furniture,
                "dining_tables": budget_input.num_dining_tables
            }
        }

    # Get recommendations
    result = get_home_recommendations(budget_input)

    # Save to history
    ✓ save_to_history(
        username=current_user.username,
        recommendation_type="home",
        input_data=budget_input.dict(),
        result=result
    )

    return result
```

- Party Planner (/generate-party): Processes party details and guest count to recommend venue, food, and decoration items

```
# Update the party-budget endpoint
@app.post("/party-budget")
async def plan_party_budget(
    budget_input: PartyBudgetInput,
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Generate party budget recommendations"""
    # Store last budget planning in session data
    if current_user.username in active_sessions:
        active_sessions[current_user.username].user_data["last_party_budget"] = {
            "timestamp": datetime.utcnow().isoformat(),
            "budget": budget_input.total_budget,
            "party_type": budget_input.party_type,
            "guests": budget_input.num_guests
        }

    # Get recommendations
    result = get_party_recommendations(budget_input)

    # Save to history
    save_to_history(
        username=current_user.username,
        recommendation_type="party",
        input_data=budget_input.dict(),
        result=result
    )

    return result
```

- Jewelry Planner (/generate-jewelry): Uses text and optional image inputs to suggest jewelry based on outfit and occasion.

```

# Update the jewelry-budget endpoint
@app.post("/jewelry-budget")
async def plan_jewelry_budget(
    total_budget: float = Form(...),
    occasion: str = Form(...),
    preferences: Optional[str] = Form(None),
    image: Optional[UploadFile] = File(None),
    request: Request = None,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Generate jewelry budget recommendations with optional outfit image"""
    # Create the budget input model
    budget_input = JewelryBudgetInput(
        total_budget=total_budget,
        occasion=occasion,
        preferences=preferences
    )

    image_path = None
    if image:
        image_path = save_upload_file(image)

    # Store last budget planning in session data
    if current_user.username in active_sessions:
        active_sessions[current_user.username].user_data["last_jewelry_budget"] = {
            "timestamp": datetime.utcnow().isoformat(),
            "budget": budget_input.total_budget,
            "occasion": budget_input.occasion,
            "has_image": image is not None
        }

```

```

# Get recommendations
result = get_jewelry_recommendations(budget_input, image_path)

# Save to history with image info
input_data = budget_input.dict()
if image:
    input_data["image"] = image.filename

save_to_history(
    username=current_user.username,
    recommendation_type="jewelry",
    input_data=input_data,
    result=result
)

return result

@app.get("/recommendation-history")
async def get_recommendation_history(
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Get the user's recommendation history"""
    if current_user.username not in user_recommendations:

```

```

    return {"history": []}

# Sort history by timestamp (newest first)
history = sorted(
    user_recommendations[current_user.username],
    key=lambda x: x.timestamp,
    reverse=True
)

# Convert to dict for JSON response
history_data = []
for item in history:
    history_data.append({
        "id": item.id,
        "timestamp": item.timestamp,
        "type": item.recommendation_type,
        "input": item.input_summary,
        "summary": item.result_summary
    })

return {"history": history_data}

```

- User Auth (/register, /login, /logout, /token): Manages user registration, authentication, and session control.

- Session Info (/session-info, /session-data): Retrieves user-specific session data for personalized AI interaction.

● Activity 3.2: Modular Architecture Setup

/recommendations-details: Returns detailed AI-generated product recommendations based on user budget, preferences, and selected category (Home, Party, or Jewelry).

```

# Step 6: Add an endpoint to get details for a specific recommendation
@app.get("/recommendation-details/{recommendation_id}")
async def get_recommendation_details(
    recommendation_id: str,
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Get the full details of a specific recommendation"""
    if current_user.username not in user_recommendations:
        raise HTTPException(status_code=404, detail="No recommendations found")

    # Find the recommendation with the given ID
    for item in user_recommendations[current_user.username]:
        if item.id == recommendation_id:
            return {
                "id": item.id,
                "timestamp": item.timestamp,
                "type": item.recommendation_type,
                "input": item.input_summary,
                "full_result": item.full_result
            }

    raise HTTPException(status_code=404, detail="Recommendation not found")

```

- **Activity 3.3: CORS & Static Routing**

/history: Retrieves the user's past recommendation queries and results for review or re-use.

```
# Step 7: Add a new endpoint to serve the history page
@app.get("/history", response_class=HTMLResponse)
async def history_page(request: Request, current_user: UserInDB = Depends(get_current_active_user)):
    """History page to view past recommendations"""
    return templates.TemplateResponse("history.html", {"request": request, "user": current_user})
```

- **Activity 3.4: Startup and main function**

/startup: Initializes essential application services and loads configuration settings when the FastAPI server starts.

```
@app.on_event("startup")
async def setup_session_cleanup():
    """Background task to clean up expired sessions"""
    async def cleanup_expired_sessions():
        while True:
            current_time = datetime.utcnow()
            # Check for sessions that have been inactive for more than 30 minutes
            expired_sessions = [
                username for username, session in active_sessions.items()
                if (current_time - session.last_activity).total_seconds() > 1800 # 30 minutes
            ]

            # Remove expired sessions
            for username in expired_sessions:
                if username in active_sessions:
                    print(f"Removing expired session for {username}")
                    del active_sessions[username]

            # Wait for 5 minutes before checking again
            await asyncio.sleep(300)

    # Start the background task
    asyncio.create_task(cleanup_expired_sessions())
```

`__main__`: Entry point of the FastAPI application that starts the server using `uvicorn` when

```
# Main entry point
if __name__ == "__main__":
    import uvicorn
    print("Starting PocketSmart: AI Budget Planner...")
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

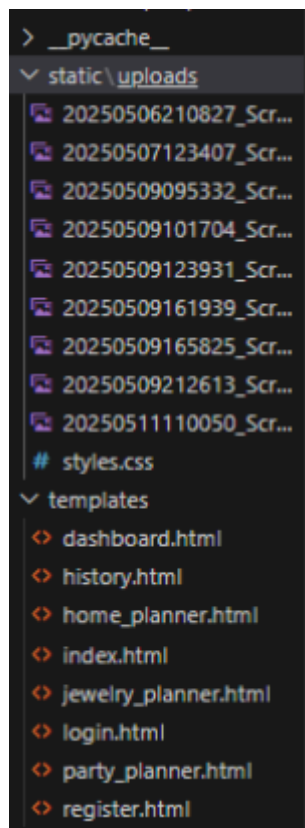
MILESTONE 4: UI Development

This milestone focuses on developing a **lightweight, responsive, and intuitive frontend** for **PocketSmart AI** using **HTML**, **CSS**, and **Jinja2 templates**. The interface enables users to interact seamlessly with the system's budget-based recommendation capabilities across Home Interior, Party Planning, and Jewelry categories. It is tightly integrated with FastAPI backend endpoints to facilitate real-time input and AI-powered output rendering. The goal is to maintain a clean, user-friendly design that supports multi-category inputs, displays structured AI recommendations, and includes authentication and session-based personalization.

Activity 4.1: Develop HTML Frontend Interface

- Design a home page with budget input fields and category selectors (Home, Party, Jewelry)
- Implement individual forms for each planner module connected to FastAPI POST routes
- Display AI responses in well-structured, card-like layouts (e.g., product suggestions, item details, prices)

Templates Directory Structure:



MILESTONE 5: Testing & Optimization

This milestone focuses on validating the reliability, accuracy, and contextual awareness of PocketSmart AI's recommendation engine. The system is tested using real-world user scenarios across different planning categories—Home Interiors, Party Planning, and Jewelry Suggestions—to ensure consistent performance. Optimization efforts include refining AI prompt structures, improving session handling, and ensuring a smooth UI/UX across

devices.

Activity 5.1: Test with Real-World Budget Inputs Across Domains Conduct end-to-end testing using varied budgets and user preferences for:

- Home decor setups from platforms like IKEA, Amazon

- Party packages including food (Swiggy/Zomato), venues (OYO), and decor
- Jewelry suggestions matched with outfit styles, occasions, and image inputs

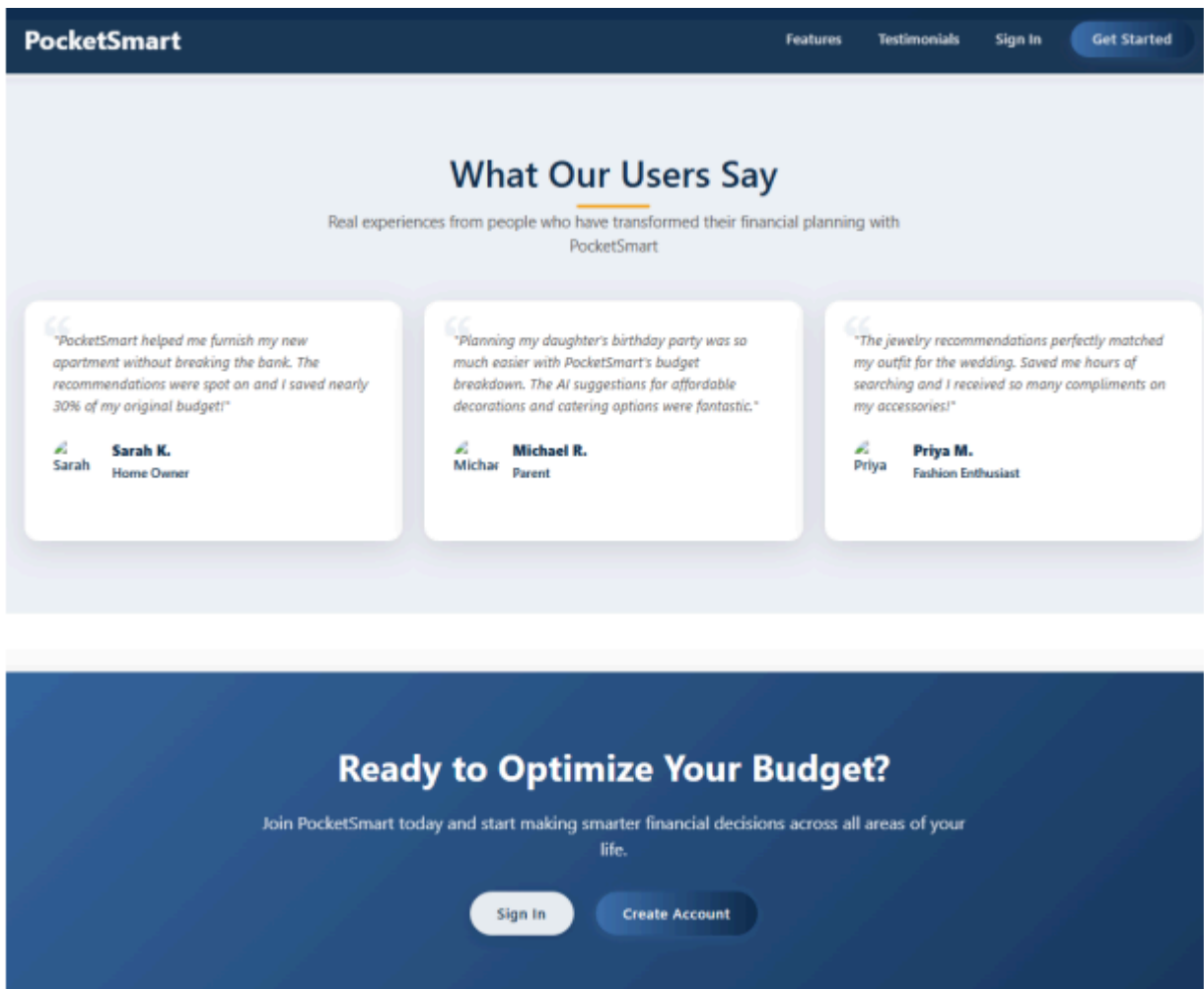
● Home Page

Home Page: The main landing page introducing PocketSmart AI's features with links to planner modules and user action.

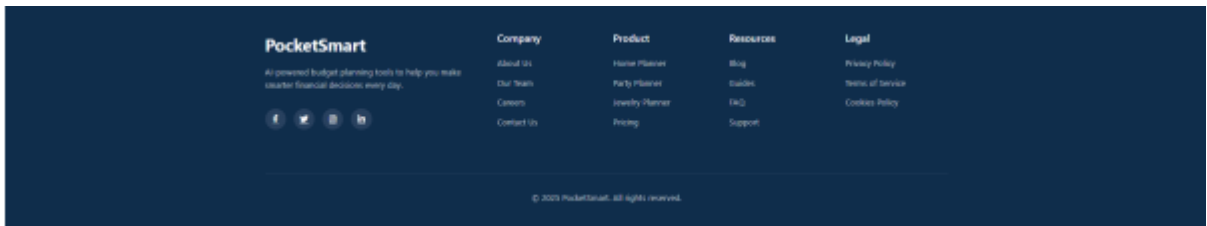


● Testimonials Page

Testimonials Page: Showcases real user reviews and success stories to demonstrate the platform’s effectiveness.



● Footer Part



● Register Page

Register Page: Allows new users to create an account to access personalized budget recommendations.

PocketSmart

AI-Powered Budget Planning

Create Your Account

Username

 Choose a username

Email

 Enter your email

Password

 Create a strong password

Confirm Password

 Confirm your password

Create Account 

Already have an account? [Sign In](#)

• Login Page

PocketSmart

AI-Powered Budget Planning

Welcome Back

Username

 Enter your username

Password

 Enter your password

[Forgot Password?](#)

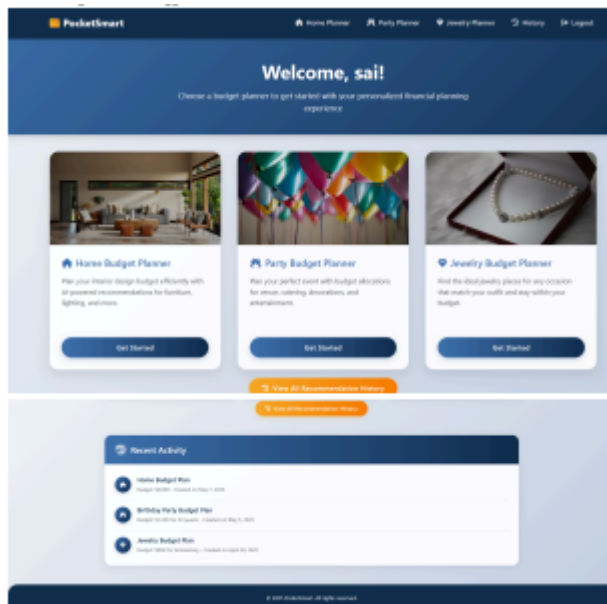
Sign In →

Don't have an account? [Create Account](#)

Login Page: Authenticates existing users to access their dashboard and saved data.

• User Dashboard

User Dashboard: Displays recent recommendations, saved queries, and personalized insights for the logged-in user.



Home Interior Budget Planner Page: Lets users input home decor preferences and budget to get tailored suggestions from multiple platforms.

The screenshot shows the "Home Interior Budget Planner" form. The title is "Home Interior Budget Planner" with a subtitle "Create a customized budget plan for your dream home interior." The form is divided into several sections: "Budget Details" with a "Total Budget (\$)" field set to "5000"; "Furniture & Furnishings" with fields for "Number of Light Fixtures" (1), "Number of Ceiling Fans" (4), "Number of Furniture Pieces" (2), and "Number of Dining Tables" (1); "Rooms to Include" with checkboxes for "Living Room", "Kitchen", and "Bedroom"; and "Additional Information" with a text area for "Special Requirements or Preferences...". A "Generate Recommendations" button is at the bottom.

Home Interior Recommendations Page: Displays AI-generated product suggestions for home decor based on the user's budget and style preferences.

Generate Recommendations

Your Personalized Budget Plan

Budget Summary

Total Budget: \$5000.00

Remaining Budget: \$500.00

Lighting

Allocation: \$1500.00

Item	Description	Price	Quantity	Shopping Links
LED Bulbs (Warm White)	Energy-efficient LED bulbs for general lighting.	\$300.00	5	Amazon Wayfair eBay Myntre Ajo

Ceiling fans

Allocation: \$2000.00

Item	Description	Price	Quantity	Shopping Links
Revolvet Ceiling Fan	Basic, functional ceiling fan.	\$500.00	4	Amazon Wayfair eBay Myntre Ajo

Furniture

Allocation: \$1000.00

Item	Description	Price	Quantity	Shopping Links
Plastic Chair	Stackable plastic chairs for kitchen or living rooms.	\$250.00	2	Amazon Wayfair eBay Myntre Ajo
Small Wooden Table	Simple wooden table for dining or side table.	\$500.00	1	Amazon Wayfair eBay Myntre Ajo

Additional Suggestions

- Consider purchasing used furniture for further cost savings.
- Look for sales and discounts on online marketplaces.
- Prioritize essential items and postpone non-essential purchases.

Party Budget Planner Page: Helps users plan events by suggesting venues, food, and decorations based on budget and guest count

PocketSmart

Home PlannerParty PlannerWedding PlannerLibraryLogout

Party Budget Planner

Plan your perfect event with AI-powered budget recommendations.

Basic Information

Total Budget (\$)

Number of Guests

1000

1

Event Details

Party Type

Venue Type

Meeting

Home

Party Needs

Catering

Decorations

Entertainment

Additional Requirements

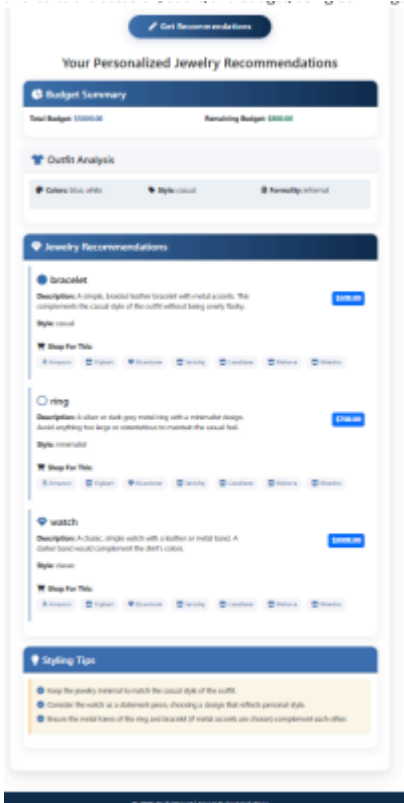
Special requests, themes, dietary restrictions, etc.

Generate Budget Plan

© 2023 PocketSmart. All rights reserved.

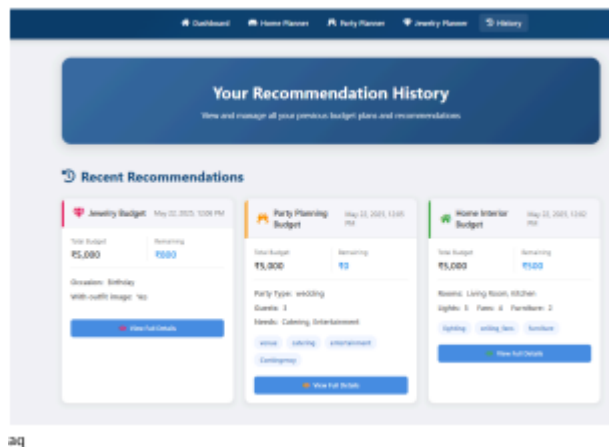
Party Budget Recommendations Page: Shows curated party planning options, including food, venues, and decorations, matched to the provided budget

Jewelry Budget Recommendations Page: Presents personalized jewelry options tailored to the occasion, outfit, and budget, using Gemini-generated insights.



Recommendation History Page: Displays a log of the user's past recommendation queries and results, allowing easy review and reuse of previous plans.

Aq



Conclusion

PocketSmart AI redefines the way individuals plan budgets for everyday lifestyle needs by delivering smart, AI-driven recommendations across **home interiors**, **party planning**, and **jewelry selection**. By combining the power of **FastAPI** with **Gemini 1.5 Flash Pro**, the system intelligently processes user inputs—budgets, preferences, and even images—to generate accurate and context-aware suggestions from multiple trusted platforms like **Amazon**, **Flipkart**, **IKEA**, **Swiggy**, **Zomato**, **OYO** and **more**. From backend architecture to AI integration and a responsive frontend, PocketSmart AI has been built with a focus on **efficiency**, **scalability**, and **user-centric design**. The FastAPI backend ensures modular, secure routing and robust session handling, while the Jinja2-powered frontend offers an intuitive and clean interface for users to interact with the system in real time.

Each module—from registration to recommendations—is designed to simplify decision-making and enhance the shopping or planning experience through automation and personalization. Whether it's finding the right wall art within budget or organizing a last-minute party, PocketSmart AI acts as a **smart digital assistant**, saving time and effort while optimizing outcomes. With its seamless fusion of generative AI and web technologies, **PocketSmart AI** stands as a practical example of how **AI can enhance everyday decision-making**, bringing intelligent recommendations to users' fingertips—budget-friendly, fast, and personalized.